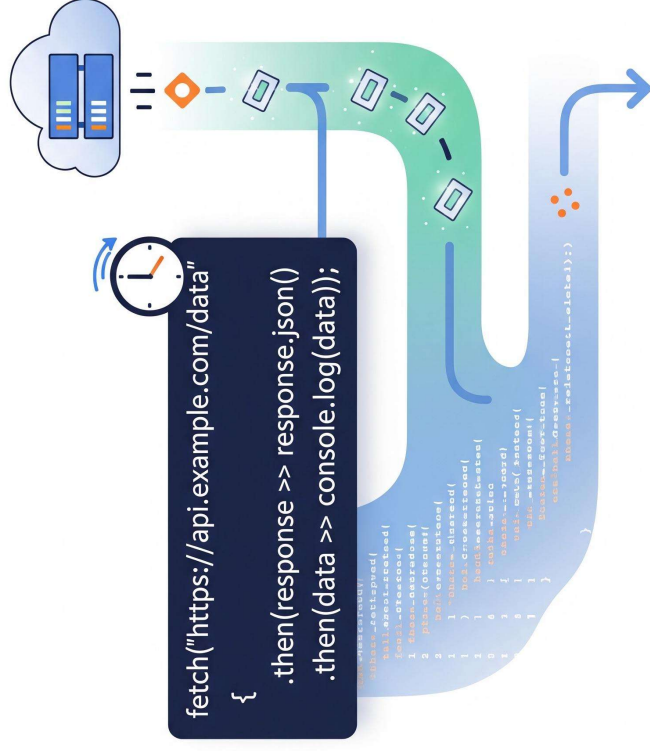


AULA 4/4

JS: Promises



O que é?

Um objeto que representa o sucesso ou a falha eventual de uma operação assíncrona

Como funciona?

- **Pendência:** A operação ainda não terminou.
- **Resolvida:** Sucesso! O valor está disponível.
- **Rejeitada:** A operação falhou.

Vantagem

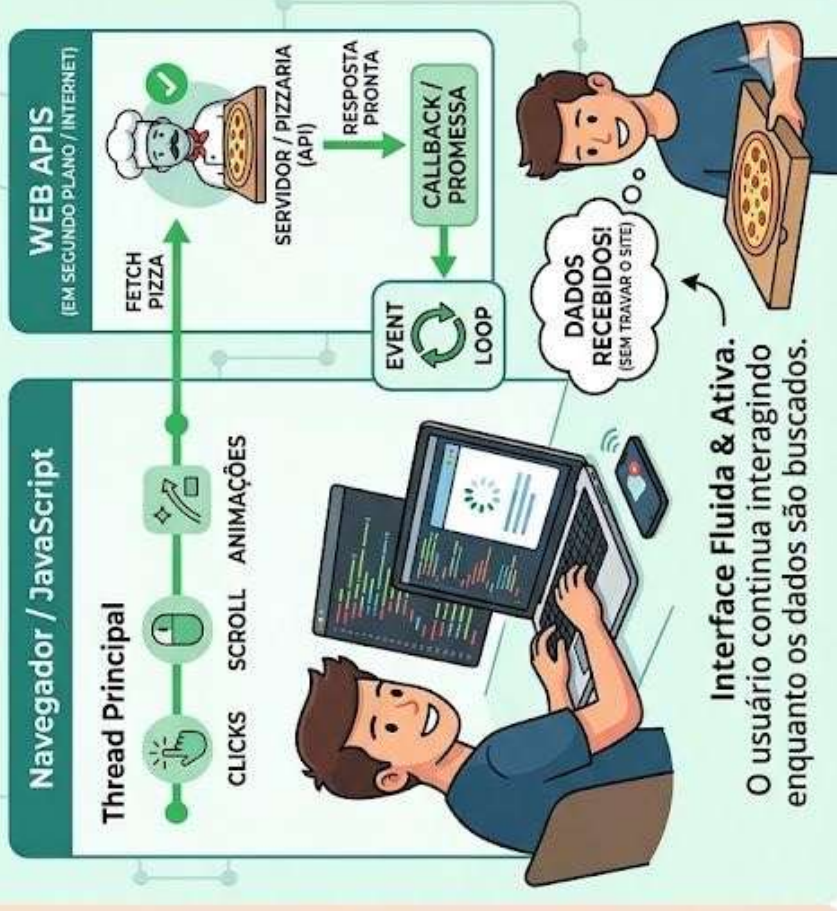
Permite anexar "handlers" (then/catch) para lidar com o resultado sem travar o código.

JS: Promises

JAVASCRIPT SÍNCRONO (O PROBLEMA)



JAVASCRIPT ASSÍNCRONO (A SOLUÇÃO)



- Faremos uma solicitação HTTP para o servidor.
- Em uma solicitação HTTP, enviamos uma mensagem para um servidor remoto, e ele nos retorna uma resposta.
- Neste caso, enviaremos uma solicitação para obter um arquivo JSON do servidor.

JS: Promises Exemplo de Uso

```
const fetchPromise = fetch(  
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",  
);  
  
console.log(fetchPromise);  
  
fetchPromise.then((response) => {  
  console.log(`Received response: ${response.status}`);  
});  
  
console.log("Started request...");
```

1. Chamando a API **fetch()** e atribuindo o valor retornado à variável **fetchPromise**.
2. Registrando no console o valor da variável **fetchPromise**. Isso deve exibir algo como: **Promise {<state>: "pending" }**, indicando que temos um objeto **Promise**, e ele possui um estado com valor "pending". O estado "pending" significa que a operação de fetch ainda está em andamento.
3. Passando uma função manipuladora para o método **then()** da **Promise**. Quando (e se) a operação de fetch for bem-sucedida, a promise chamará nosso manipulador, passando um objeto **Response**, que contém a resposta do servidor.
4. Registrando uma mensagem no console indicando que a solicitação foi iniciada.

- Com a API `fetch()`, assim que você obtém um objeto `Response`, é necessário chamar outra função para obter os dados da resposta.
- Neste caso, queremos obter os dados da resposta como JSON, então chamamos o método `json()` do objeto `Response`.
- Acontece que o método `json()` também é assíncrono. Portanto, este é um caso em que precisamos chamar duas funções assíncronas sucessivas.

JS: Promises Exemplo de Uso

```
const fetchPromise = fetch(
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
);

fetchPromise.then((response) => {
  const jsonPromise = response.json();
  jsonPromise.then((data) => {
    console.log(data[0].name);
  });
});
```

- Neste exemplo, como antes, adicionamos um manipulador `then()` à promise retornada pelo `fetch()`. Mas, desta vez, nosso manipulador chama `response.json()` e, em seguida, passa um novo manipulador `then()` para a promise retornada por `response.json()`.
- Isso deve registrar no console "baked beans" (o nome do primeiro produto listado em "products.json").

- Lembra quando dissemos que ao chamar um callback dentro de outro callback acabávamos criando níveis sucessivamente mais aninhados de código?
- E dissemos que esse "inferno dos callbacks" tornava nosso código difícil de entender? Isso não é exatamente a mesma coisa, só que com chamadas ao `then()`?
- De fato, é. Mas a característica elegante das promises é que o próprio `then()` retorna uma promise, que será resolvida com o resultado da função passada para ele.
- Isso significa que podemos (e certamente devemos) reescrever o código.

JS: Promises Exemplo de Uso

```
const fetchPromise = fetch(
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
);

fetchPromise
  .then((response) => response.json())
  .then((data) => {
    console.log(data[0].name);
  });
```

- Em vez de chamar o segundo `then()` dentro do manipulador do primeiro `then()`, podemos retornar a promise retornada por `json()` e chamar o segundo `then()` nesse valor de retorno.
- Isso é chamado de encadeamento de promises (promise chaining) e nos permite evitar níveis cada vez maiores de indentação quando precisamos fazer chamadas consecutivas a funções assíncronas.

Por fim ...

```
const fetchPromise = fetch(  
  "bad-scheme://mdn.github.io/learning-area/javascript/apis/fetching-data/can-  
  store/products.json",  
);  
  
fetchPromise  
  .then((response) => {  
    if (!response.ok) {  
      throw new Error(`HTTP error: ${response.status}`);  
    }  
    return response.json();  
  })  
  .then((data) => {  
    console.log(data[0].name);  
  })  
  .catch((error) => {  
    console.error(`Could not get products: ${error}`);  
  });
```

- Há uma forma ainda mais elegante de trabalhar com código assíncrono no Javascript, ao invés de usar a notação convencional para Promises, vamos utilizar o padrão Async e Await.
- A palavra-chave `async` oferece uma maneira mais simples de trabalhar com código assíncrono baseado em promises. Adicionar `async` no início de uma função a torna uma função assíncrona:

```
async function myFunction() {  
    // This is an async function  
}
```

- Dentro de uma função async, você pode usar a palavra-chave await antes de uma chamada para uma função que retorna uma promise.
- Isso faz com que o código espere nesse ponto até que a promise seja resolvida, momento em que o valor resolvido da promise é tratado como um valor de retorno, ou o valor rejeitado é lançado como um erro.
- Isso permite que você escreva código que utiliza funções assíncronas, mas que se parece com código síncrono. Por exemplo, poderíamos usá-la para reescrever nosso exemplo de fetch.

Exemplos de uso

- <https://hp-api.onrender.com/>
- API IBGE
- API Prefeitura Quixadá

```

async function fetchAllCharacters() {
  try {
    const response = await fetch('https://hp-api.onrender.com/api/characters');
    if (!response.ok) {
      throw new Error(`Erro HTTP: ${response.status}`);
    }
    const characters = await response.json();
    console.log("Personagens:", characters);
    return characters;
  } catch (error) {
    console.error("Falha ao buscar personagens:", error);
  }
}

async function getDataa(){
  const allChars = await fetchAllCharacters();
  console.log("Primeiro personagem:", allChars[0].name);
}

getDataa();

```

```

async function fetchCharacterById(id) {
  try {
    const response = await fetch(`https://hp-api.onrender.com/api/character/${id}`);
    if (!response.ok) {
      throw new Error(`Personagem não encontrado (ID: ${id})`);
    }
    const character = await response.json();
    console.log("Personagem encontrado:", character);
    return character;
  } catch (error) {
    console.error("Erro:", error.message);
  }
}

async function getData(){
  const char = await fetchCharacterById("9e3f7ce4-b9a7-4244-b709-dae5c1f1d4a8"); // ID de
  Harry Potter
  console.log(char[0].name);
}

getData();

```



```
async function fetchGryffindorStudents() {
  try {
    const response = await
    fetch('https://hp-api.onrender.com/api/characters/house/gryffindor');
    const students = await response.json();
    console.log("Alunos da Grifinória:", students);

    // Filtra apenas alunos vivos (opcional)
    const livingStudents = students.filter(student => !student.alive);
    console.log("Alunos vivos:", livingStudents);
  } catch (error) {
    console.error("Erro ao buscar alunos:", error);
  }
}

// Uso
fetchGryffindorStudents();
```

```
async function fetchSpells() {
  try {
    const response = await fetch('https://hp-api.onrender.com/api/spells');
    const spells = await response.json();

    console.log("Feitiços disponíveis:");
    spells.forEach(spell => {
      console.log(` - ${spell.name}: ${spell.description}`);
    });

    return spells;
  } catch (error) {
    console.error("Erro ao buscar feitiços:", error);
  }
}

// Uso
fetchSpells();
```

- . Crie um sistema para baixar os dados da URL abaixo e exibir, em formato de card, os dados dos personagens, inclusive a imagem.
- . Se o personagem não tiver imagem, exiba uma imagem padrão.

URL com os dados: <https://hp-api.onrender.com/api/characters/staff>



www.shutterstock.com · 744867163